LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Rifqi Ramadhan (2609106040)

Kelas (B1‘26)

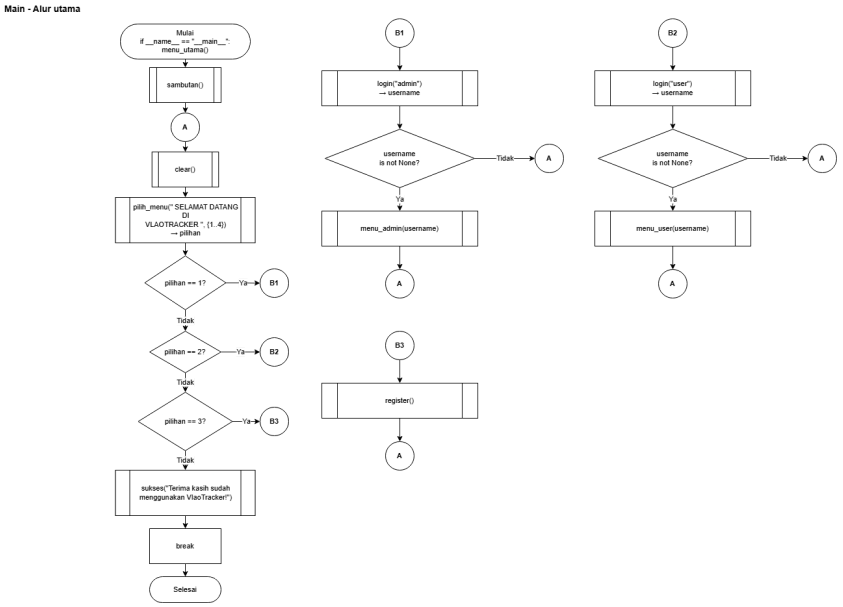
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

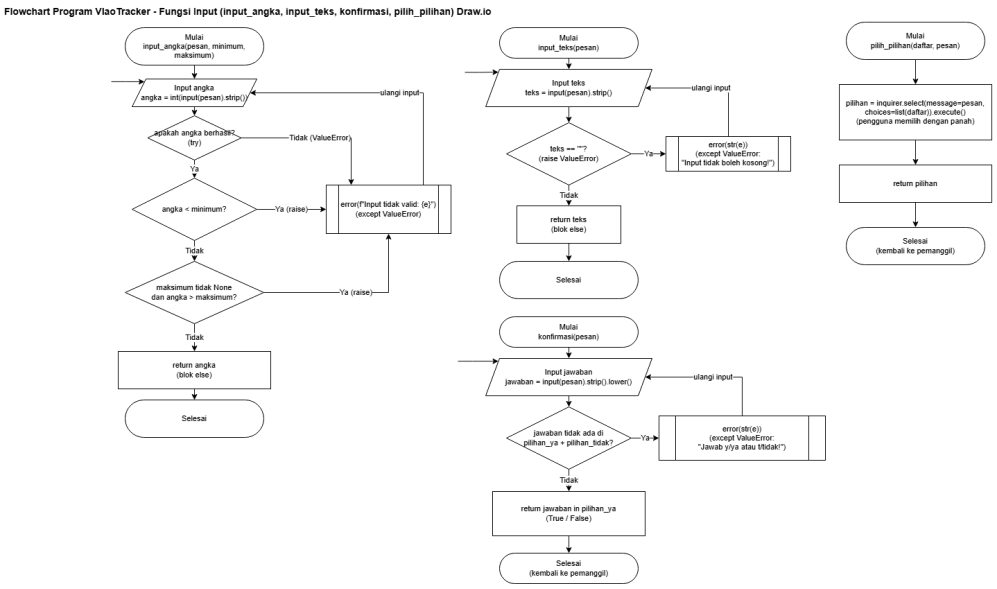
1. Flowchart



Gambar 1.1 Flowchart Program VlaoTracker - Alur Utama Program (Main) Draw.io

Alur utama program **VlaoTracker** (menu_utama) dimulai dengan menampilkan pesan sambutan pembuka kepada pengguna. Program kemudian memasuki menu utama yang punya empat pilihan: masuk sebagai Admin (pilihan 1), masuk sebagai Pengguna/User (pilihan 2), melakukan registrasi akun baru (pilihan 3), atau keluar dari program (pilihan 4).

Jika pengguna memilih opsi Admin atau User, sistem akan memverifikasi proses login terlebih dahulu, jika valid, sistem akan mengarahkan pengguna ke menu manajemen admin (menu_admin) atau dasbor pengguna (menu_user), dan jika gagal, alur akan kembali ke menu utama. Sementara itu, opsi registrasi akan menjalankan fungsi pendaftaran akun baru sebelum kembali ke menu utama. Program akan terus berulang hingga pengguna memilih opsi keluar, di mana sistem menampilkan pesan penutup dan menghentikan eksekusi program di titik *Selesai*.



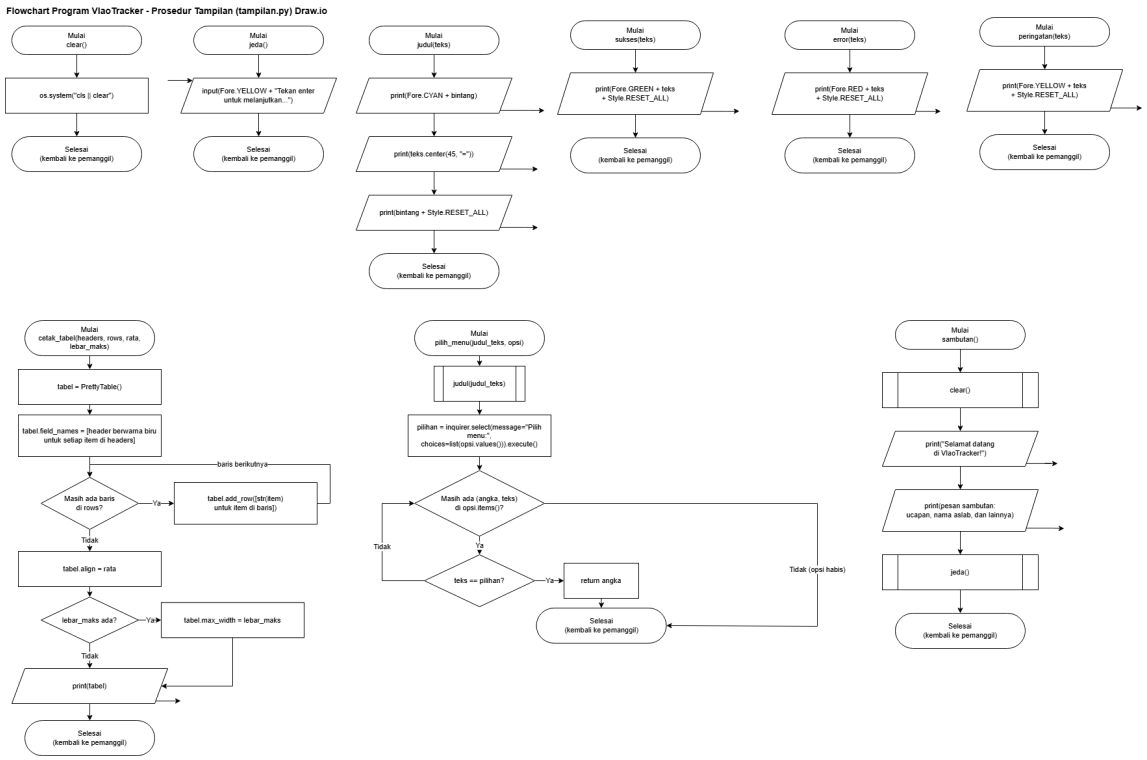
Gambar 1.2 Flowchart Program VlaoTracker - Fungsi input_angka (Error Handling) Draw.io

input_angka(pesan, minimum, maksimum): Menerima input angka dari pengguna, memvalidasi apakah input berupa bilangan bulat yang valid, serta memastikan nilai berada dalam batas minimum dan maksimum yang ditentukan sebelum mengembalikan nilai angka. Jika tidak valid, program menampilkan pesan error dan mengulang proses input.

input_teks(pesan): Meminta input teks dari pengguna dan memastikan bahwa teks yang dimasukkan tidak kosong (""). Jika kosong, program memunculkan peringatan dan meminta pengguna memasukkan ulang teks tersebut.

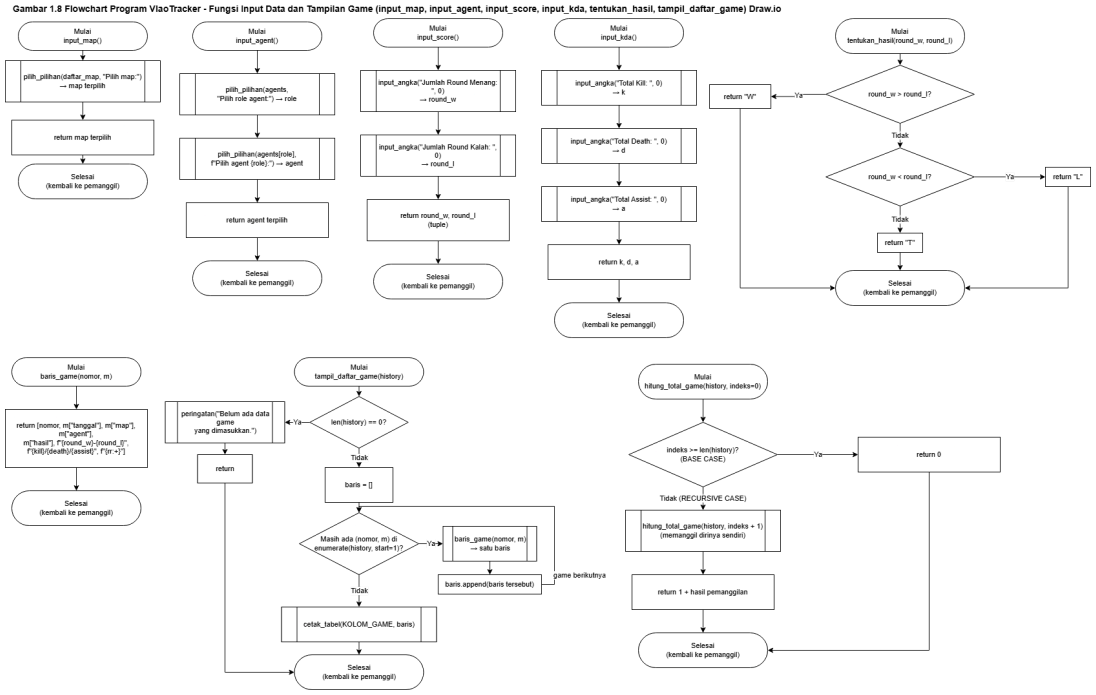
konfirmasi(pesan): Meminta jawaban konfirmasi dari pengguna, memeriksa apakah input sesuai dengan pilihan ya atau tidak, lalu mengembalikan nilai boolean (True atau False).

pilih_pilihan(daftar, pesan): Pakai library interaktif (inquirer.select) untuk menampilkan daftar opsi kepada pengguna agar dapat memilih menggunakan tombol panah, lalu mengembalikan pilihan yang dipilih.



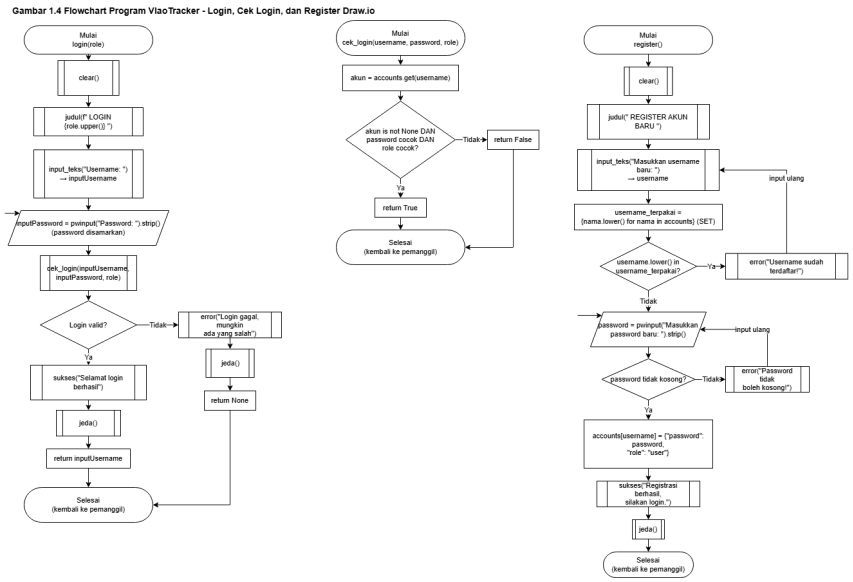
Gambar 1.3 Flowchart Program VlaoTracker - Login, Cek Login, dan Register Draw.io

clear(): Berfungsi untuk membersihkan layar terminal menggunakan perintah sistem operasi (cls atau clear), **jeda()**: Menampilkan pesan kuning agar pengguna menekan tombol Enter untuk melanjutkan program, **judul(teks)**: Mencetak judul atau tajuk program dengan format dekoratif menggunakan karakter khusus di tengah layar, **sukses(teks)**, **error(teks)**, & **peringatan(teks)**: Mencetak pesan status kepada pengguna dengan warna yang berbeda-beda (hijau untuk sukses, merah untuk galat/error, dan kuning untuk peringatan), **cetak_tabel(headers, rows, rata, lebar_maks)**: Menginisialisasi objek PrettyTable, memasukkan baris-baris data dan kepala tabel, mengatur perataan teks serta lebar maksimum, lalu mencetak tabel yang rapi ke layar, **pilih_menu(judul_teks, opsi)**: Menampilkan judul menu dan memfasilitasi pemilihan opsi interaktif menggunakan panah pemilih (*inquirer*), lalu mengembalikan nomor angka pilihan pengguna, **sambutan()**: Membersihkan layar, menampilkan pesan sambutan awal program VlaoTracker beserta informasi ucapan atau asisten laboratorium, dan menjeda eksekusi sebelum masuk ke menu utama.



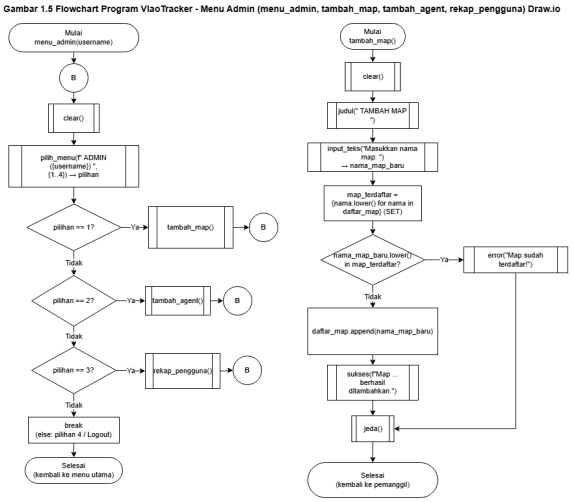
Gambar 1.4 Flowchart Program VlaoTracker - Menu Admin Draw.io

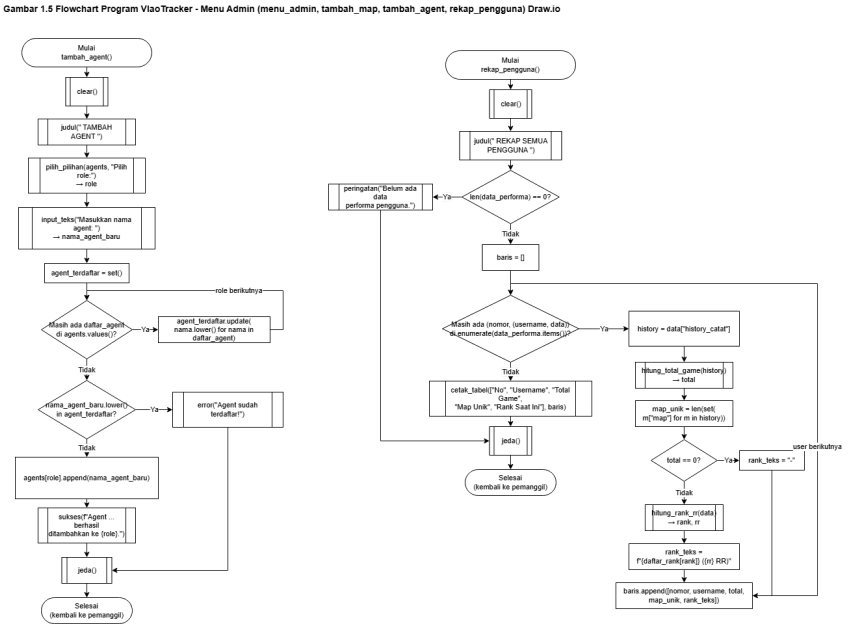
input_map(): Meminta pengguna memilih map permainan dari daftar pilihan yang tersedia dan mengembalikan map yang terpilih, **input_agent()**: Meminta pengguna memilih role terlebih dahulu, lalu memilih agen spesifik berdasarkan role tersebut untuk dikembalikan ke pemanggil, **input_score()**: Meminta input angka untuk jumlah round menang dan round kalah, kemudian mengembalikan kedua nilai tersebut dalam bentuk tuple, **input_kda()**: Meminta input angka untuk total Kill, Death, dan Assist, lalu mengembalikan ketiga nilai statistik KDA tersebut, **tentukan_hasil(round_w, round_l)**: Membandingkan jumlah round menang dan kalah untuk menentukan hasil pertandingan, mengembalikan "W" jika menang, "L" jika kalah, dan "T" jika seri, **baris_game(nomor, m)**: Memformat data suatu pertandingan (tanggal, map, agen, hasil, skor, dan KDA) menjadi satu baris data untuk ditampilkan ke dalam tabel, **tampil_daftar_game(history)**: Memeriksa apakah riwayat game kosong; jika ada, fungsi mengonversi setiap data game ke dalam baris tabel dan mencetaknya menggunakan cetak_tabel, **hitung_total_game(history, indeks)**: Menggunakan metode rekursif untuk menghitung total keseluruhan game yang tercatat di dalam riwayat (history).



Gambar 1.5 Flowchart Program VlaoTracker - Dashboard User dan Menu CRUD Draw.io

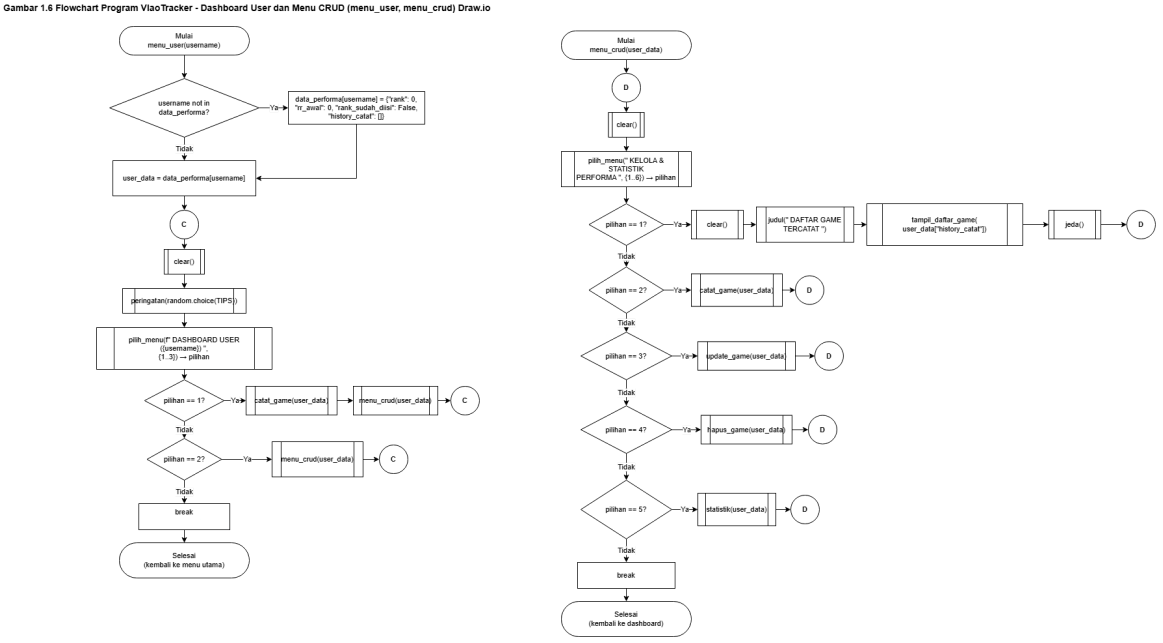
login(role): Berfungsi untuk mengelola proses masuk pengguna dengan membersihkan layar, mencetak judul, meminta input username serta password tersembunyi (masked), lalu memvalidasi kredensial melalui fungsi pengecekan untuk mengembalikan username yang valid atau None jika gagal, **cek_login(username, password, role)**: Berfungsi untuk memeriksa kecocokan data akun di dalam sistem dengan memverifikasi keberadaan akun, kecocokan password, serta kesesuaian role, lalu mengembalikan nilai boolean (True atau False), **register()**: Mengelola proses pendaftaran akun baru dengan meminta input username baru, memvalidasi ketersediaannya agar tidak duplikat, memastikan password tidak kosong, serta menyimpan data akun baru ke dalam kamus accounts dengan peran "user".





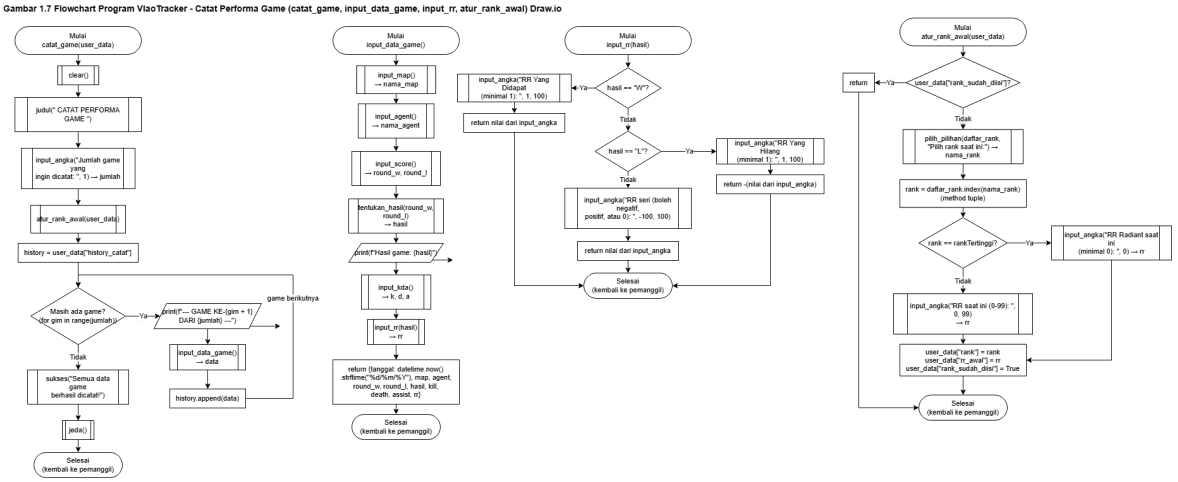
Gambar 1.6 Flowchart Program VlaoTracker - Catat Performa Game Draw.io

menu_admin(username): Berfungsi untuk menampilkan menu manajemen admin secara interaktif dengan pilihan untuk menambah map, menambah agen, melihat rekapitulasi data seluruh pengguna, atau keluar (logout) kembali ke menu utama, **tambah_map()**: Memproses input nama map baru dari administrator, memeriksa ketersediaannya di dalam sistem untuk menghindari duplikasi data, lalu menyimpan map baru ke dalam daftar atau menampilkan pesan galat jika map tersebut sudah terdaftar, **tambah_agent()**: Berfungsi untuk menambahkan agen baru ke dalam sistem dengan meminta administrator memilih role, menginput nama agen baru, memeriksa duplikasi nama, lalu menyimpannya ke dalam kategori role yang sesuai atau menampilkan pesan galat jika agen sudah terdaftar, **rekap_pengguna()**: Berfungsi untuk memeriksa ketersediaan data performa seluruh pengguna, menghitung total game secara rekursif, menghitung jumlah map unik, mengevaluasi rank terkini, lalu menyusun dan mencetak hasilnya dalam bentuk tabel rekapitulasi.



Gambar 1.6 Flowchart Program VlaoTracker - Catat Performa Game Draw.io

menu_user(username): Memeriksa apakah data performa pengguna sudah terdaftar; jika belum, sistem akan menginisialisasi struktur data dasar, lalu menampilkan tips acak dan menu interaktif untuk mencatat game atau menuju menu manajemen data CRUD, **menu_crud(user_data)**: Menyediakan menu pilihan interaktif untuk mengelola data game, yang mencakup opsi melihat daftar game tercatat, mencatat game baru (catat_game), memperbarui data game (update_game), menghapus game (hapus_game), melihat statistik performa (statistik), atau kembali ke dasbor.

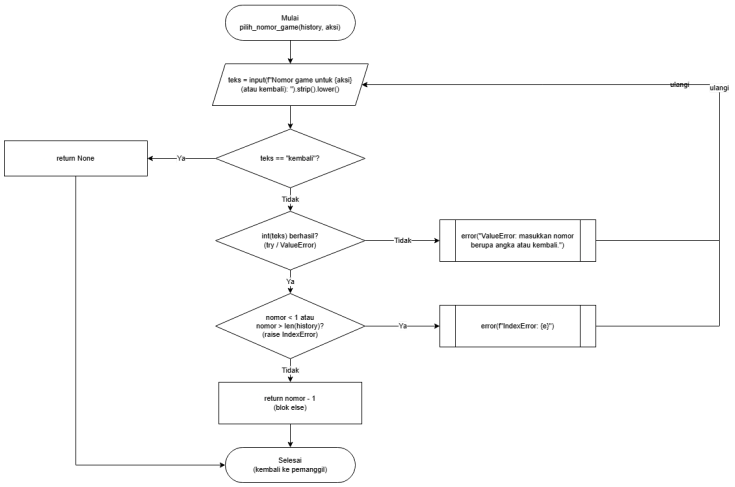


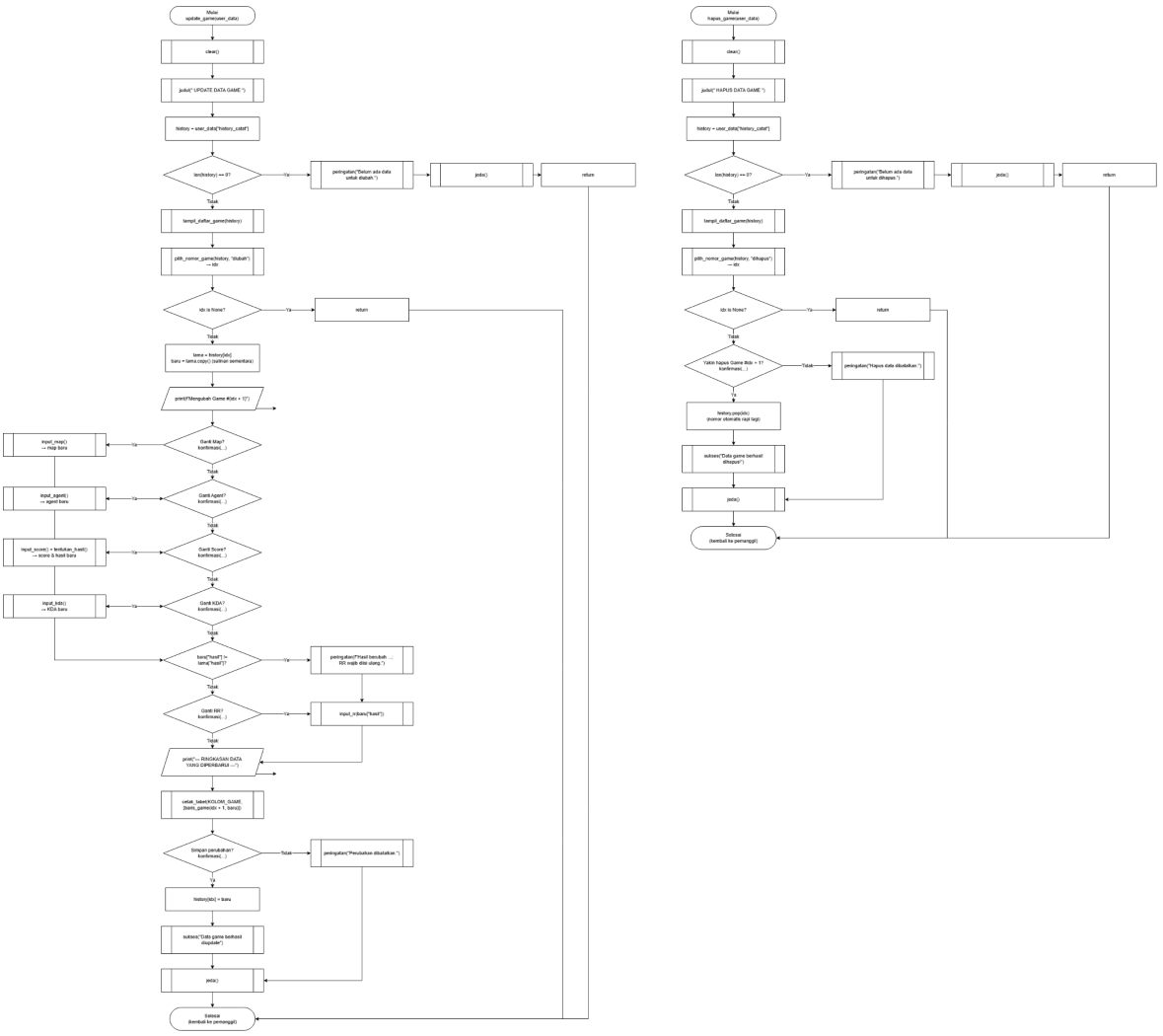
Gambar 1.8 Flowchart Program VlaoTracker - Statistik dan Perhitungan Rank Draw.io

catat_game(user_data): Mengelola proses pencatatan beberapa pertandingan sekaligus dengan meminta jumlah game, mengatur rank awal jika belum diisi, lalu melakukan perulangan untuk

mengumpulkan data setiap game dan menambahkannya ke dalam riwayat (history), **input_data_game()**: Menggabungkan proses pengumpulan data satu pertandingan penuh—yang mencakup pemilihan map, pemilihan agen, skor round, penentuan hasil pertandingan, KDA, dan perolehan RR—untuk kemudian dikembalikan dalam bentuk struktur data kamus/dictionary,**input_rr(hasil)**: Menyesuaikan input perolehan Rank Rating (RR) berdasarkan hasil pertandingan, di mana nilai akan positif jika menang ("W"), negatif jika kalah ("L"), atau bernilai bebas jika seri ("T"), **atur_rank_awal(user_data)**: Memeriksa apakah rank awal pengguna sudah diisi; jika belum, fungsi ini meminta pengguna memilih rank awal dan RR saat ini, lalu menandai status pengisian rank menjadi True.

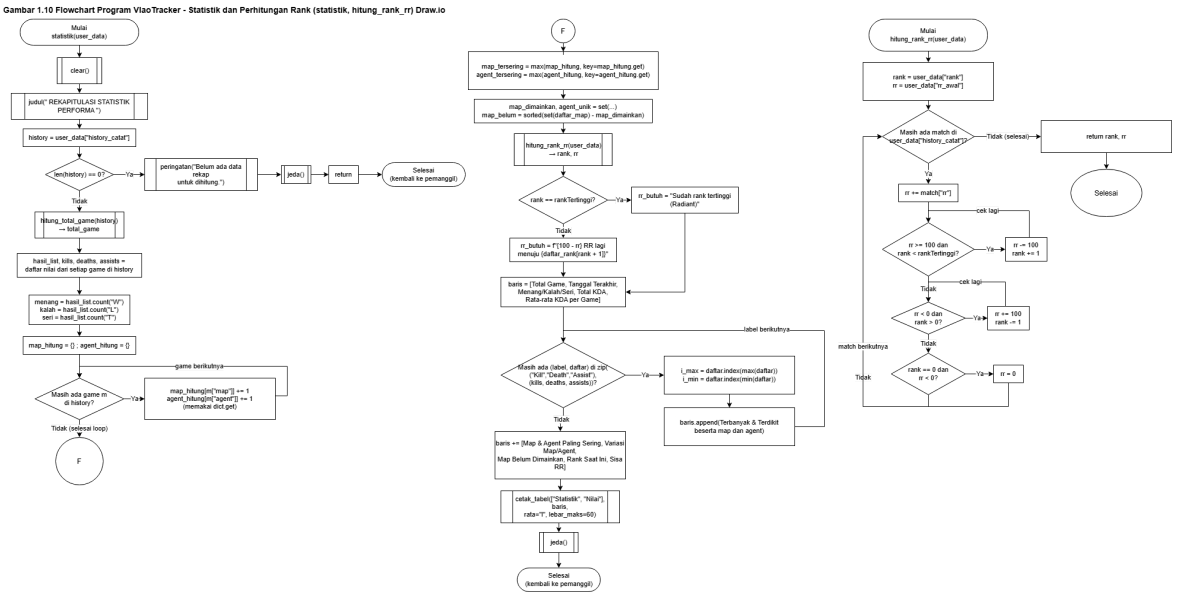
Gambar 1.9 Flowchart Program ViaoTracker - Update dan Hapus Data Game (pilih_nomor_game, update_game, hapus_game) Draw.io





Gambar 1.9 Flowchart Program VlaoTracker - Fungsi Rekursif hitung_total_game Draw.io

update_game(user_data): Berfungsi untuk memperbarui data pertandingan yang telah tercatat dengan memeriksa keberadaan riwayat game, memilih nomor game yang ingin diubah, memodifikasi parameter tertentu (seperti map, agen, skor, KDA, atau RR) secara interaktif, dan menyimpan perubahan jika dikonfirmasi, **hapus_game(user_data)**: Berfungsi untuk menghapus data pertandingan dari riwayat (history) setelah pengguna memilih nomor game yang ingin dihapus dan memberikan konfirmasi persetujuan, **pilih_nomor_game(history, aksi)**: Berfungsi untuk meminta input nomor game dari pengguna, memvalidasi apakah input berupa angka yang valid serta berada dalam rentang indeks riwayat yang tersedia, menangani kesalahan (error), dan mengembalikan indeks data yang dipilih.



Gambar 1.10 Flowchart Program VlaoTracker - Fungsi Input (Predefined Process) Draw.io

statistik(user_data): Berfungsi untuk memeriksa ketersediaan riwayat game, menghitung total game, menghitung frekuensi kemenangan/kekalahan/seri, menentukan map dan agen favorit, memanggil fungsi perhitungan rank, serta menyusun dan mencetak ringkasan statistik performa ke dalam bentuk tabel, **hitung_rank_rr(user_data)**: Berfungsi untuk memproses simulasi perubahan rank dan perolehan Rank Rating (RR) secara iteratif berdasarkan riwayat pertandingan, di mana rank akan naik jika RR mencapai ≥ 100 dan turun jika RR bernilai < 0.

2. Deskripsi Singkat Program

Tujuan VlaoTracker dibuat untuk membantu pemain Valorant mencatat dan memantau performa permainan ranked.

Manfaat utama, pencatatan performa game, dapat menambah, melihat, mengubah, dan menghapus data game yang sudah tercatat, sehingga kesalahan input bisa diperbaiki, rekap otomatis disini pemain bisa mengevaluasi permainannya. Dan untuk admin bisa melihat berapa banyak yang memakai program ini.

3. Source Code

A. Menampilkan Data Dengan PrettyTable

Fitur cetak_tabel membuat tabel dari header dan baris data. Header diberi warna sama lebar dapat dibatasi.

```
from prettytable import PrettyTable
from colorama import Fore, Style

def cetak_tabel(headers, rows, rata="c", lebar_maks=None):
    tabel = PrettyTable()
    tabel.field_names = [f"{Fore.BLUE}{item}{Style.RESET_ALL}" for item in
headers]

    for baris in rows:
        tabel.add_row([str(item) for item in baris])
    tabel.align = rata
    if lebar_maks:
        tabel.max_width = lebar_maks
    print(tabel)
```

B. Fitur catat game

Fitur ini dipakai pengguna untuk mencatat satu atau beberapa game sekaligus. Program sekarang akan otomatis mencatat tanggal, meminta jumlah game, mengatur rank awal (hanya sekali), lalu memanggil input_data_game() untuk setiap

game. Nomor game dibuat dari nomor terbesar ditambah satu supaya tidak menimpa data lama

```
from datetime import datetime

def tentukan_hasil(round_w, round_l):
    if round_w > round_l:
        return "W"
    elif round_w < round_l:
        return "L"
    return "T"

def input_data_game():
    nama_map = input_map()
    nama_agent = input_agent()
    round_w, round_l = input_score()
    hasil = tentukan_hasil(round_w, round_l)
    print(f"Hasil game: {hasil}")
    k, d, a = input_kda()
    rr = input_rr(hasil)

    return {
        "tanggal": datetime.now().strftime("%d/%m/%Y"),
        "map": nama_map,
        "agent": nama_agent,
        "round_w": round_w,
        "round_l": round_l,
        "hasil": hasil,
        "kill": k,
        "death": d,
        "assist": a,
        "rr": rr,
    }

def catat_game(user_data):
    jumlah = input_angka("Jumlah game yang ingin dicatat: ",
1)
    atur_rank_awal(user_data)
    history = user_data["history_catat"]
    for gim in range(jumlah):
        print(f"\n--- GAME KE-{gim + 1} DARI {jumlah} ---")
        history.append(input_data_game())
```

```
    sukses("\nSemua data game berhasil dicatat!")
```

C. Update Data

Fitur ini mengubah data game yang sudah dicatat. Perubahan dikerjakan pada dictionary sementara (lama copy()), sehingga data asli baru berubah setelah pengguna mengonfirmasi penyimpanan.

```
 lama = history[idx]
baru = lama.copy()

if konfirmasi(f"Ganti Map ({lama['map']})? (y/t): "):
    baru["map"] = input_map()
if konfirmasi(f"Ganti Agent ({lama['agent']})? (y/t): "):
    baru["agent"] = input_agent()
if konfirmasi(f"Ganti Score
({lama['round_w']}-{lama['round_l']})? (y/t): "):
    baru["round_w"], baru["round_l"] = input_score()
    baru["hasil"] = tentukan_hasil(baru["round_w"],
baru["round_l"])

if baru["hasil"] != lama["hasil"]:
    baru["rr"] = input_rr(baru["hasil"])
elif konfirmasi(f"Ganti RR ({lama['rr']:+})? (y/t): "):
    baru["rr"] = input_rr(baru["hasil"])

if konfirmasi("Simpan perubahan? (y/t): "):
    history[idx] = baru
    sukses("Data game berhasil diupdate")
```

D. Hapus Data

Fitur ini menghapus satu game setelah pengguna mengonfirmasi. Setelah data dihapus, nomor game disusun ulang dari 1 supaya urutannya tidak berlubang.

```
def hapus_game(user_data):
    clear()
    judul(" HAPUS DATA GAME ")
    history = user_data["history_catat"]
    if len(history) == 0:
        peringatan("Belum ada data untuk dihapus.")
        jeda()
        return
```

```
    tampil_daftar_game(history)
    idx = pilih_nomor_game(history, "dihapus")
    if idx is None:
        return
    if konfirmasi(f"Yakin hapus Game #{idx + 1}? (y/t): "):
        history.pop(idx)
        sukses("Data game berhasil dihapus!")
    else:
        peringatan("Hapus data dibatalkan.")
    jeda()
```

E. Fitur Statistik dan Perhitungan Rank

Fitur ini merekap performa pengguna dalam satu tabel dan fitur untuk memperbarui RR dari setiap game.

```
hasil_list = [match["hasil"] for match in history]
kills = [match["kill"] for match in history]
deaths = [match["death"] for match in history]
assists = [match["assist"] for match in history]

menang = hasil_list.count("W")
kalah = hasil_list.count("L")
seri = hasil_list.count("T")

map_hitung = {}
agent_hitung = {}
for match in history:
    map_hitung[match["map"]] = map_hitung.get(match["map"],
0) + 1
    agent_hitung[match["agent"]] =
agent_hitung.get(match["agent"], 0) + 1

map_tersering = max(map_hitung, key=map_hitung.get)
agent_tersering = max(agent_hitung, key=agent_hitung.get)

baris = [
    ["Total Game Dimainkan", total_game],
    ["Menang / Kalah / Seri", f"{menang} / {kalah} /
{seri}"],
    ["Total KDA", f"{sum(kills)} / {sum(deaths)} /
{sum(assists)}"],
    [
```

```
        "Rata-rata KDA per Game",
        f"{round(sum(kills)/total_game, 1)} / "
        f"{round(sum(deaths)/total_game, 1)} / "
        f"{round(sum(assists)/total_game, 1)}",
    ],
    ["Map Paling Sering Main", f"{map_tersering}
({map_hitung[map_tersering]}x)"],
    ["Agent Paling Sering", f"{agent_tersering}
({agent_hitung[agent_tersering]}x)"],
]
cetak_tabel(["Statistik", "Nilai"], baris, rata="l",
lebar_maks=60)
```

```
def hitung_rank_rr(user_data):
    rank = user_data["rank"]
    rr = user_data["rr_awal"]
    for match in user_data["history_catat"]:
        rr += match["rr"]
        while rr >= 100 and rank < rankTertinggi:
            rr -= 100
            rank += 1
        while rr < 0 and rank > 0:
            rr += 100
            rank -= 1
        if rank == 0 and rr < 0:
            rr = 0
    return rank, rr
```

F. Library

PrettyTable, Colorama, inquirerPy, pwinput, datetime, random, os

```
from InquirerPy import inquirer
from pwinput import pwinput

pilihan = inquirer.select(
    message="Pilih menu:",
    choices=list(opsi.values()),
    pointer="➡️ ",
    qmark="",
).execute()

password = pwinput("Masukkan password: ").strip()
```

4. Hasil Output

```
Selamat datang di VlaoTracker!
Halo, halo, Selamat Pagi, Hidup Mahasiswa, Hidup Rakyat Indonesia, Hidup Perempuan Yang Melawan, Hidup Teknik! S
elamat datang di program paling epik abad ini: VLAOTRACKER

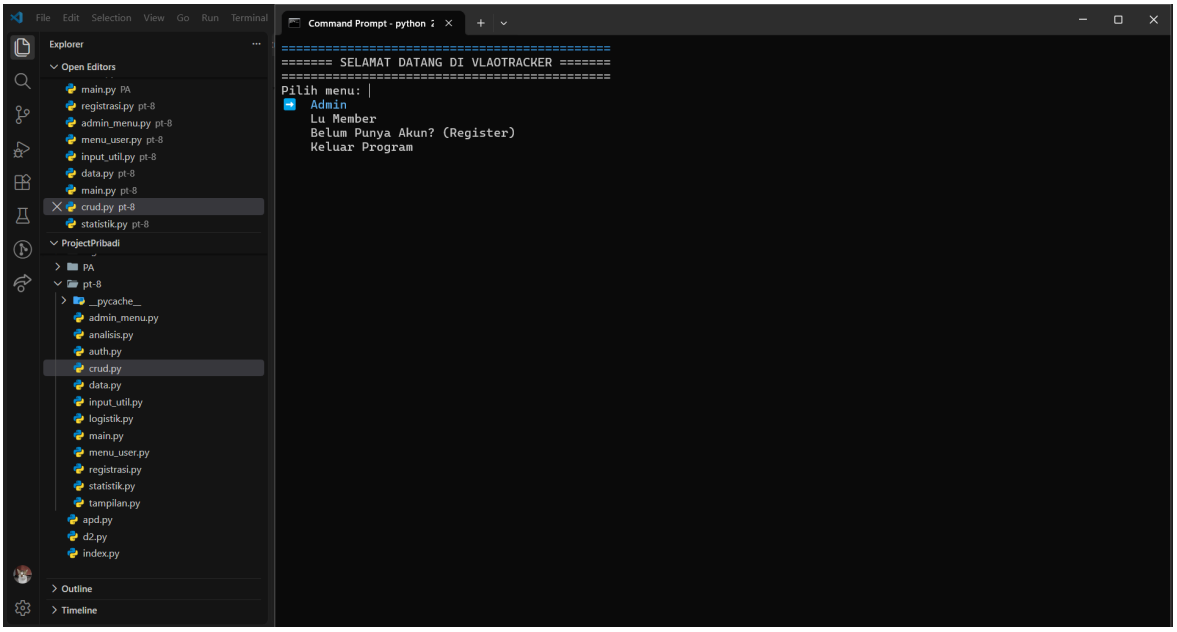
Dibuat sebelum menuju TA huhuhuhu, secangkir esteh yang membuat semangat sebentar, dan dipersembahkan untuk asla
b:
-> NLG'25 (068) (Ketua)
-> R'25   (106)
-> SNA'25 (016)
-> YAP'25 (017)
Well... Well... Well. Posttest APD terakhir:D

Program ini didedikasikan buat nilai 100 dan menambah pengetahuan.

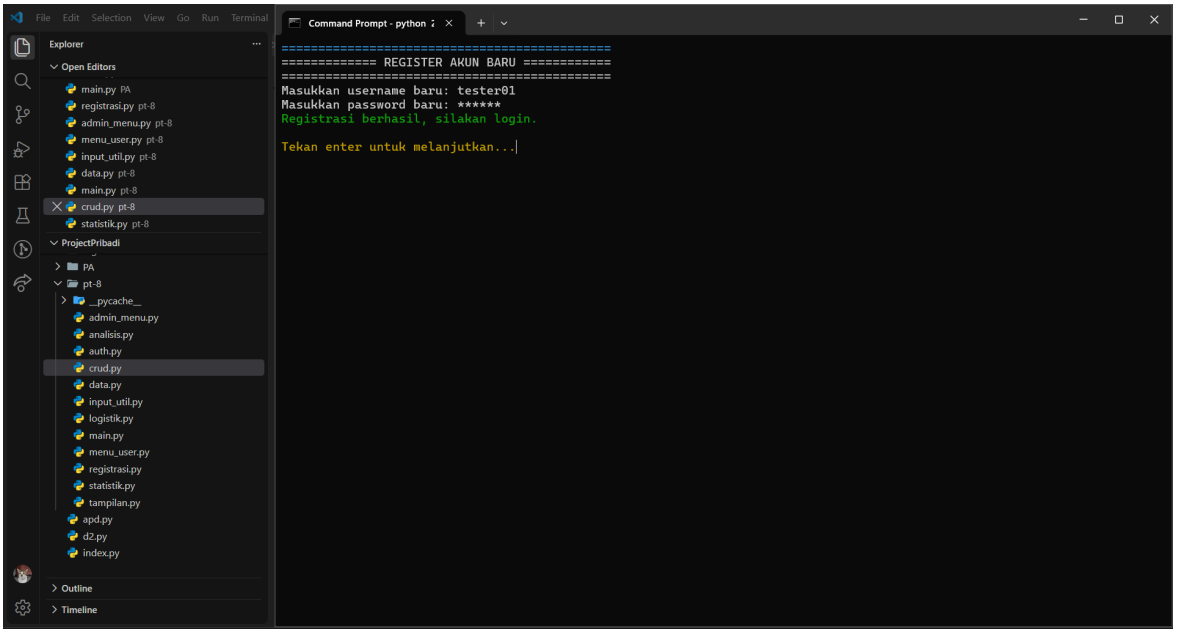
Intinya: Catat performa, tampilin stat.
semoga lulus posttest, dan jangan lupa senyum ya Aslab Aslab Aslab Aslab sibuk banget kaya penjabat guah (ermmm
ini dari stiker btw bukan saya yang ngomong :D) Yey! 🎉
time to lock in

Tekan enter untuk melanjutkan...
```

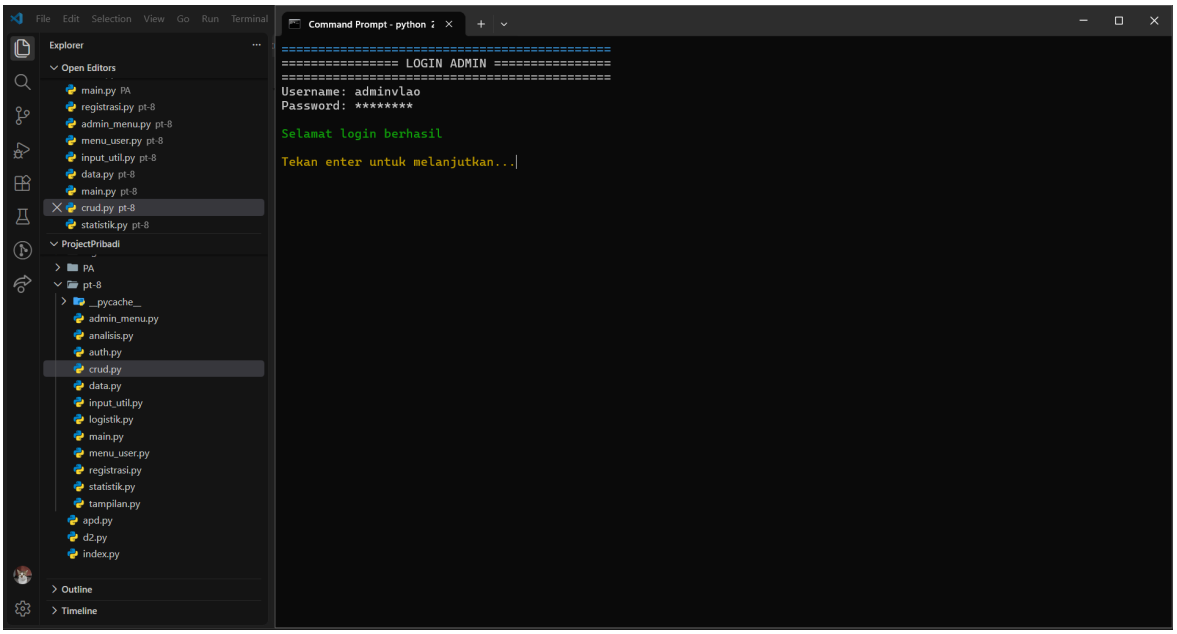
Gambar 4.1 Screenshot Output Terminal sambutan



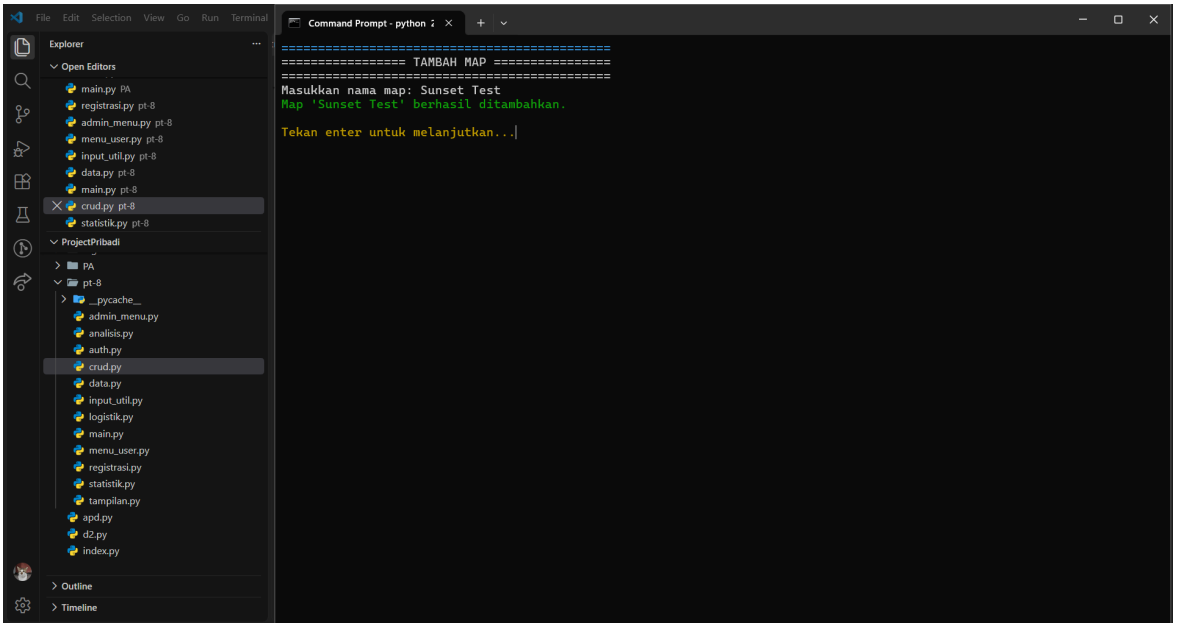
Gambar 4.2 Screenshot Output Terminal Inquirer Menu Awal



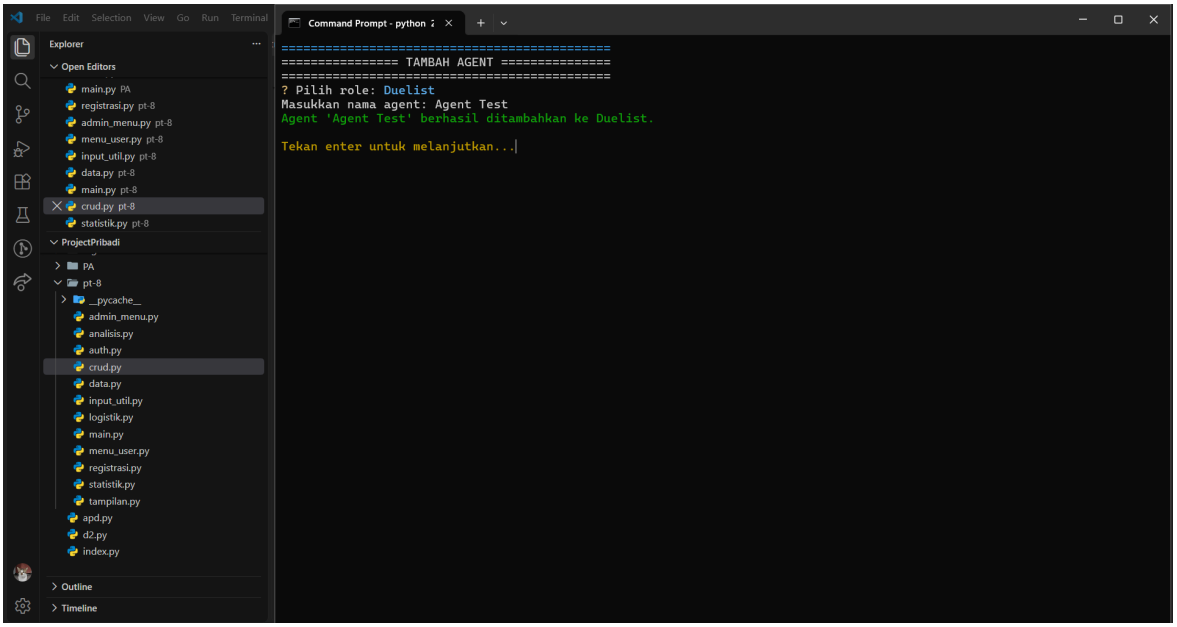
Gambar 4.3 Screenshot Output Terminal Halaman Registrasi & Registrasi Berhasil



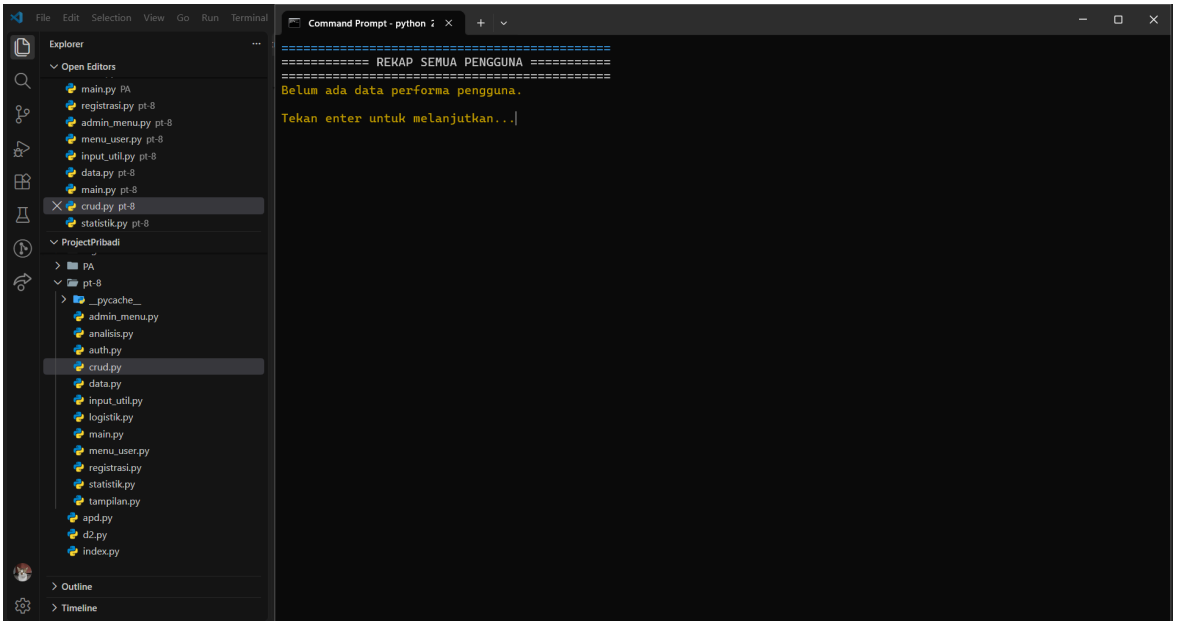
Gambar 4.4 Screenshot Output Terminal Login Berhasil



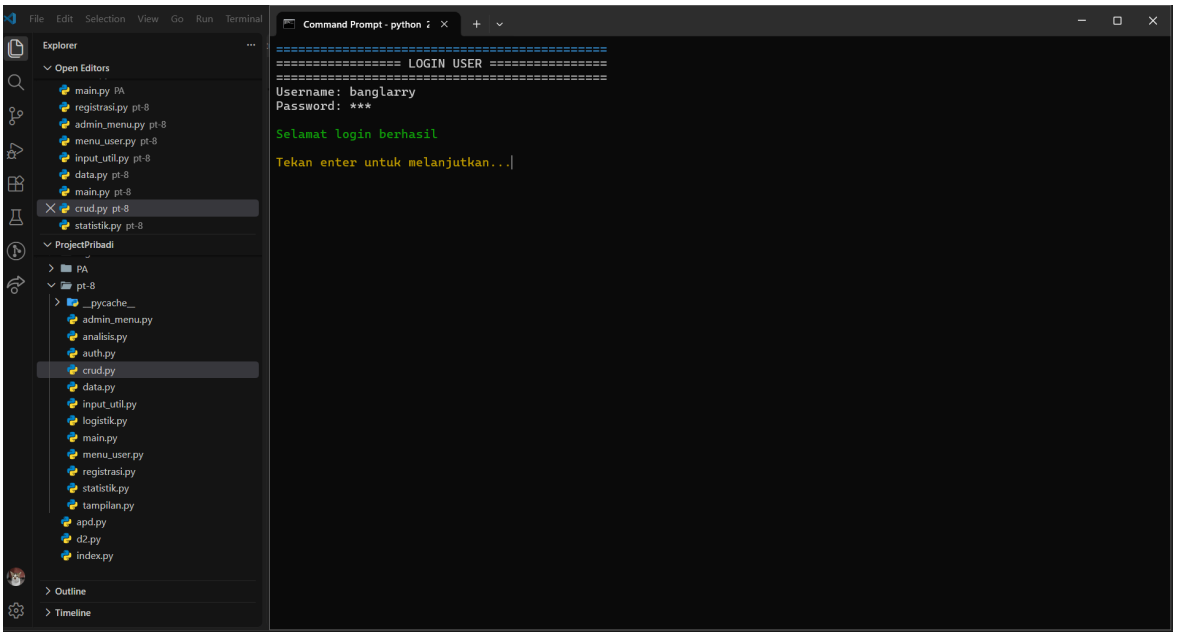
Gambar 4.5 Screenshot Output Terminal Menu Admin - Tambah Map



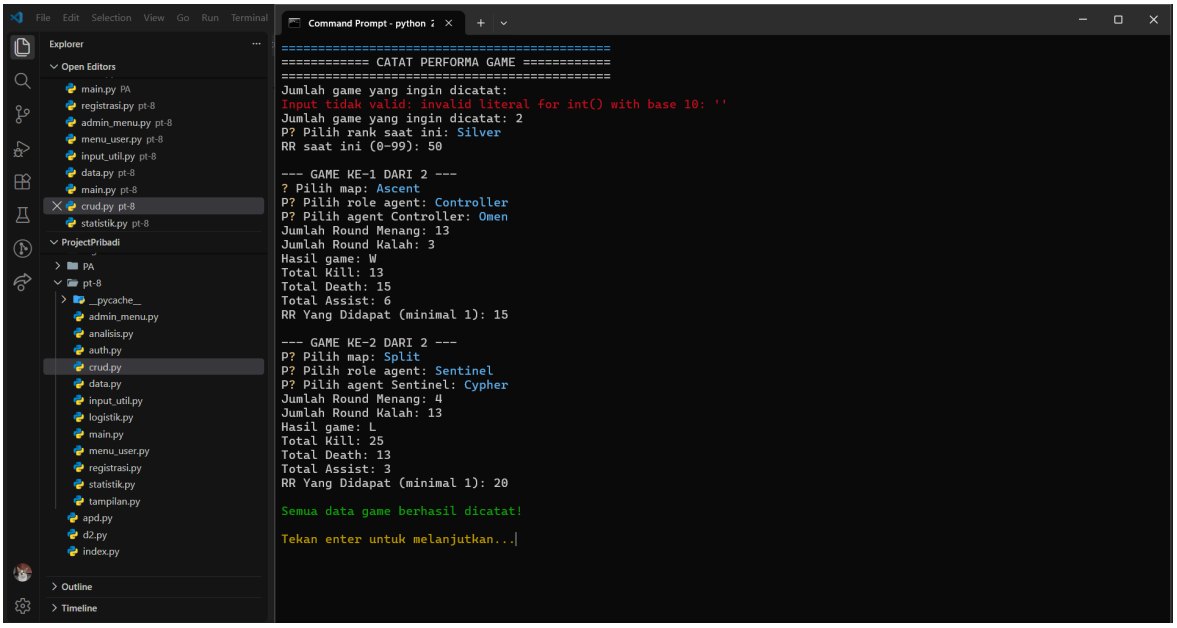
Gambar 4.6 Screenshot Output Terminal Admin Menu - Tambah Agent



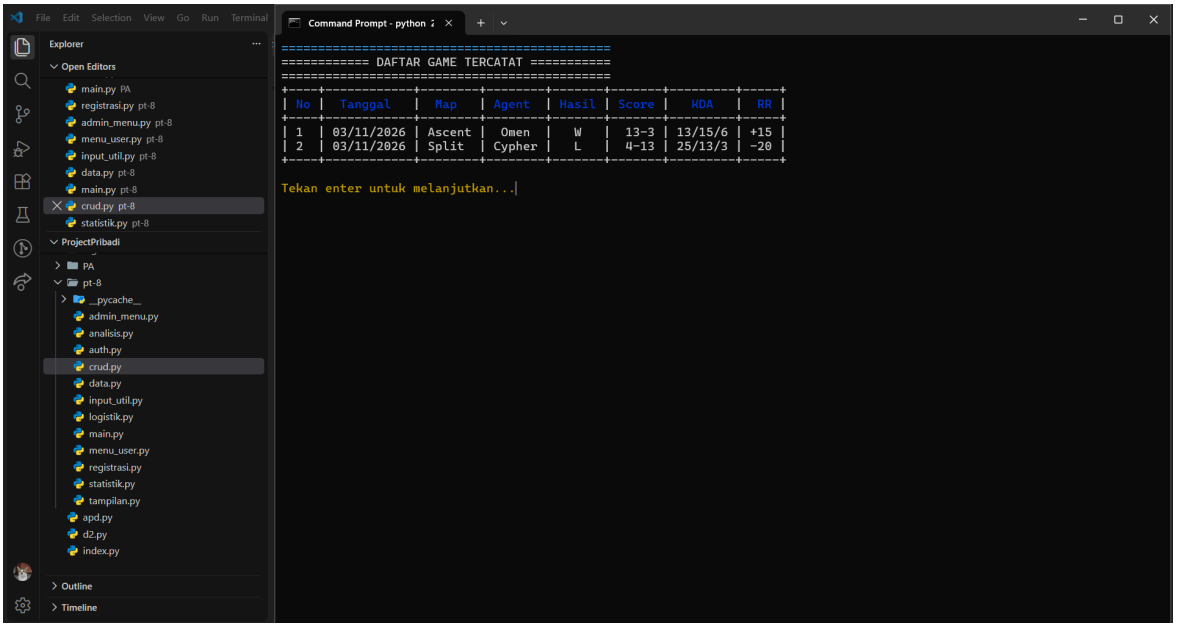
Gambar 4.7 Screenshot Output Terminal Rekap Belum Ada Data



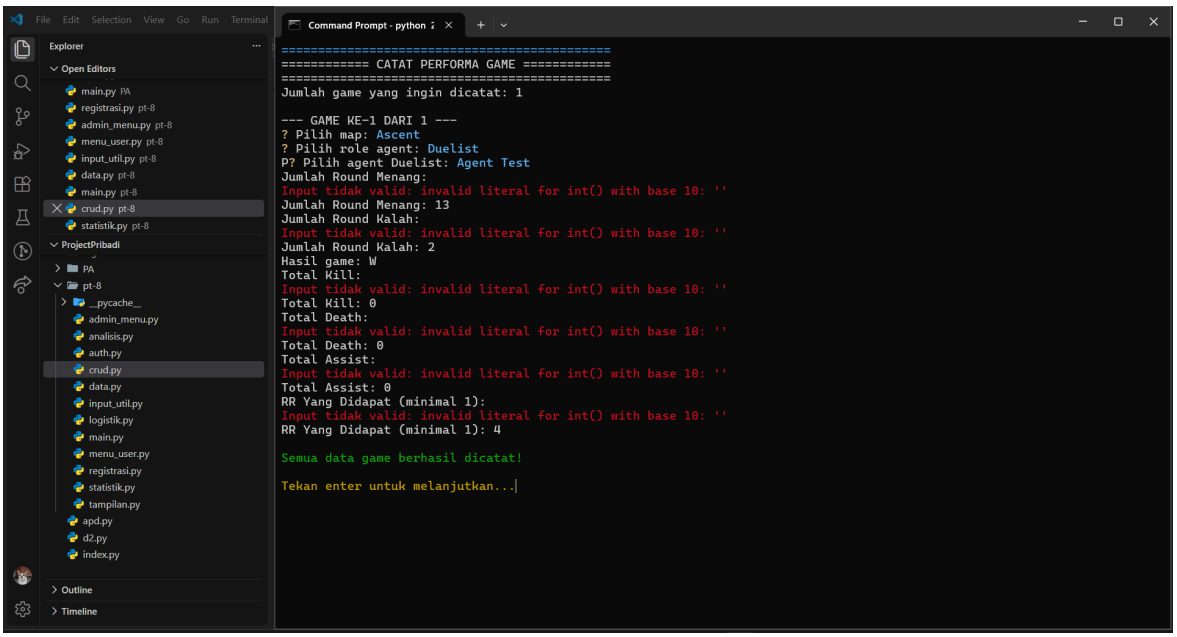
Gambar 4.8 Screenshot Output Terminal Login Berhasil



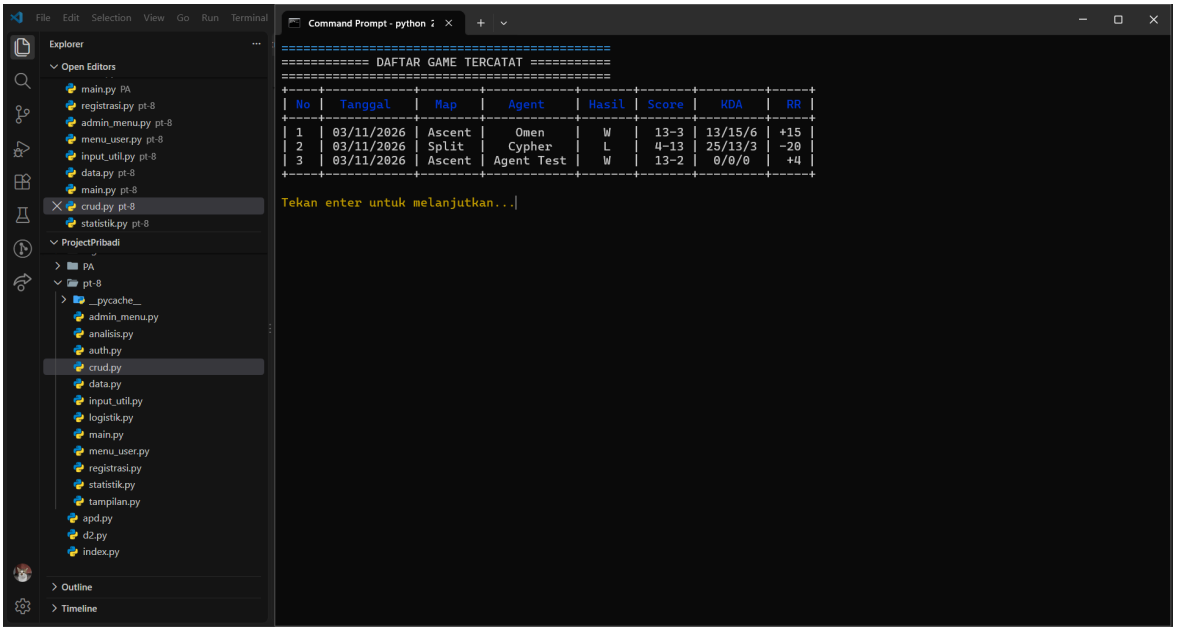
Gambar 4.8 Screenshot Output Terminal Catat Game User



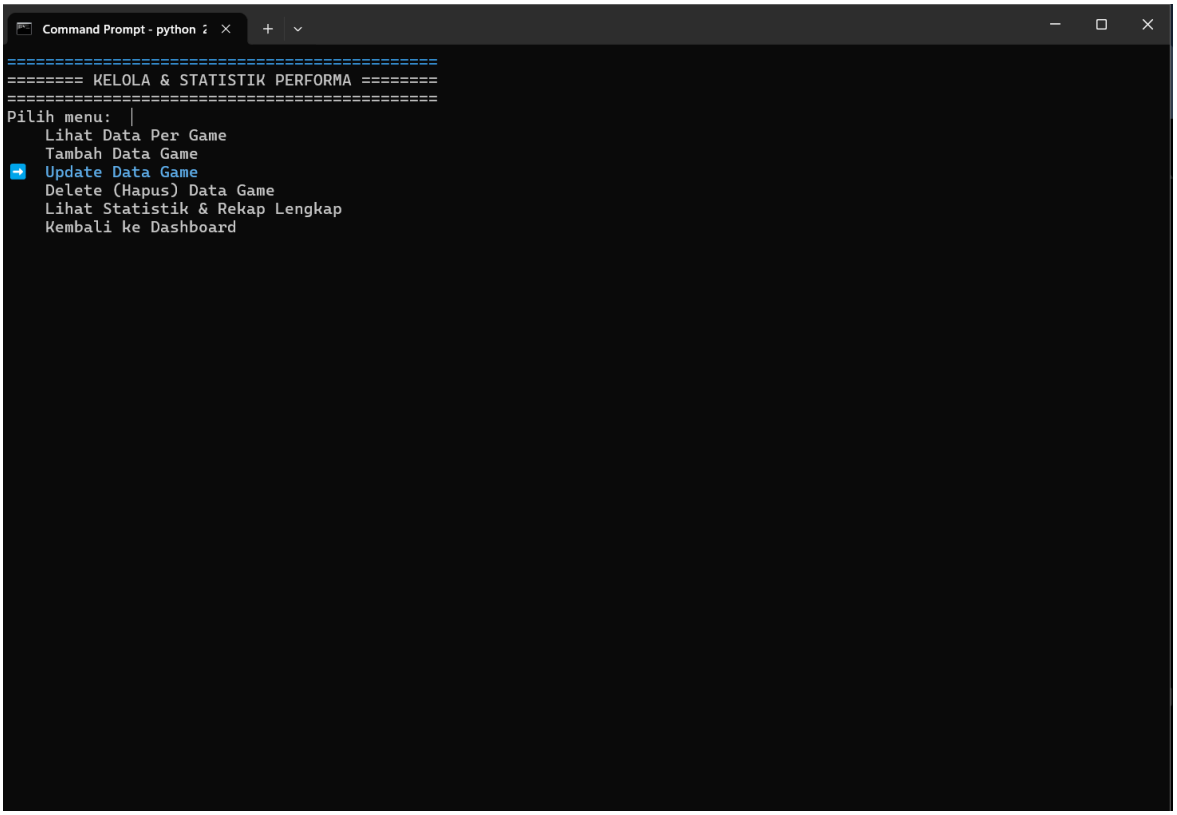
Gambar 4.9 Screenshot Output Terminal Read Awal



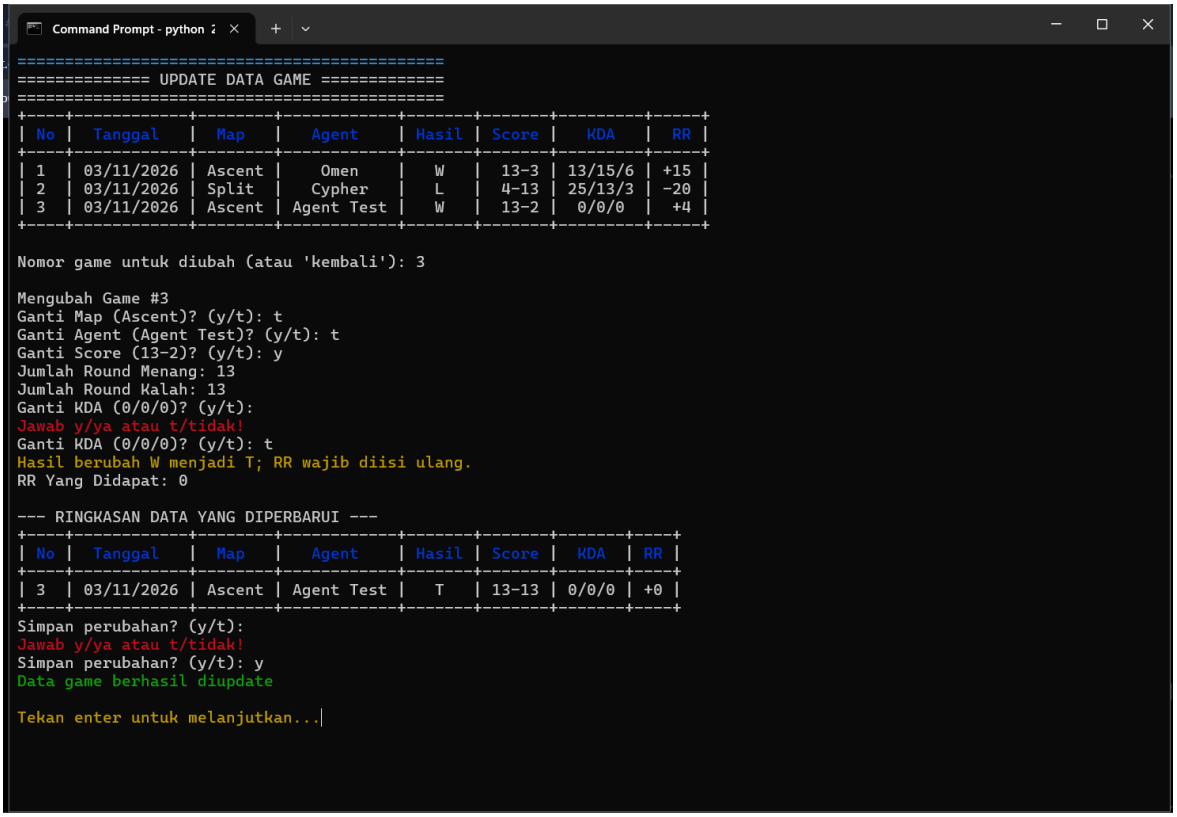
Gambar 4.10 Screenshot Output Terminal Create Data Baru dan Error Handlingnya



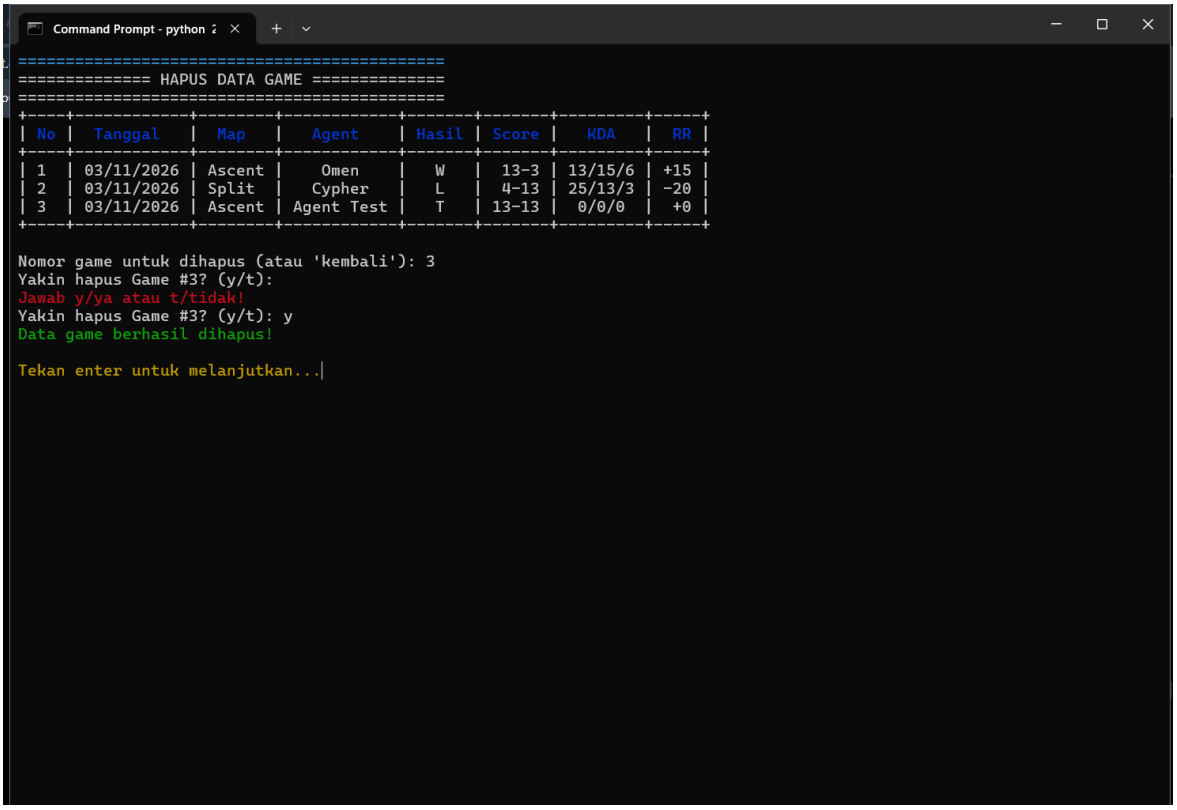
Gambar 4.11 Screenshot Output Terminal Read Setelah Create



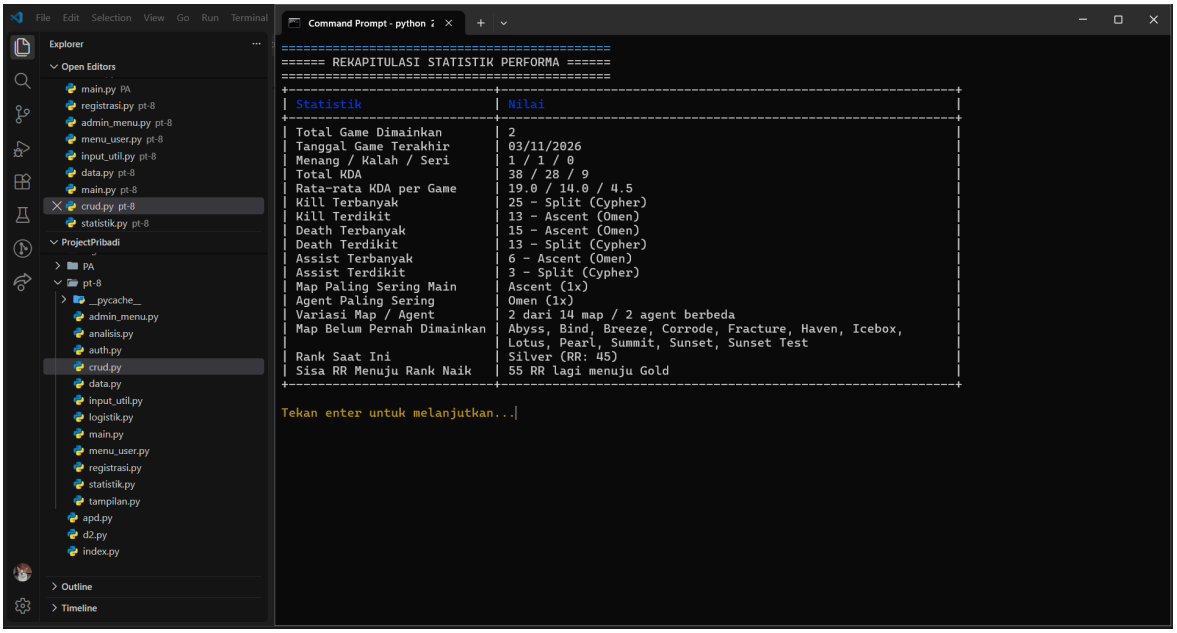
Gambar 4.12 Screenshot Output Terminal Form Menu User



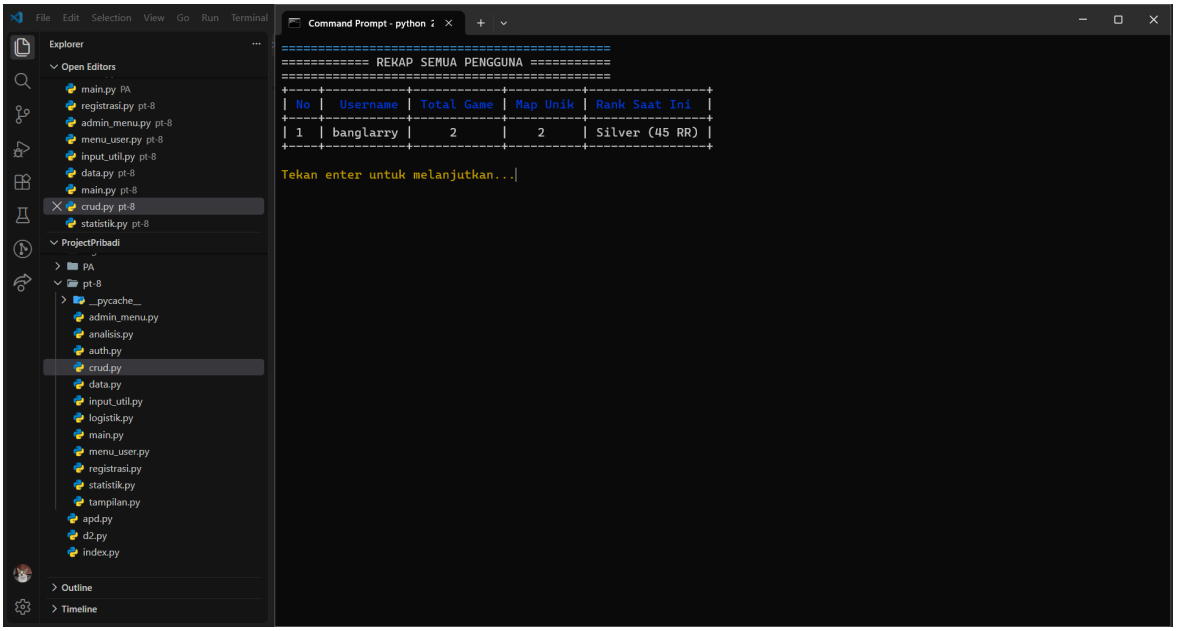
Gambar 4.13 Screenshot Output Terminal Update



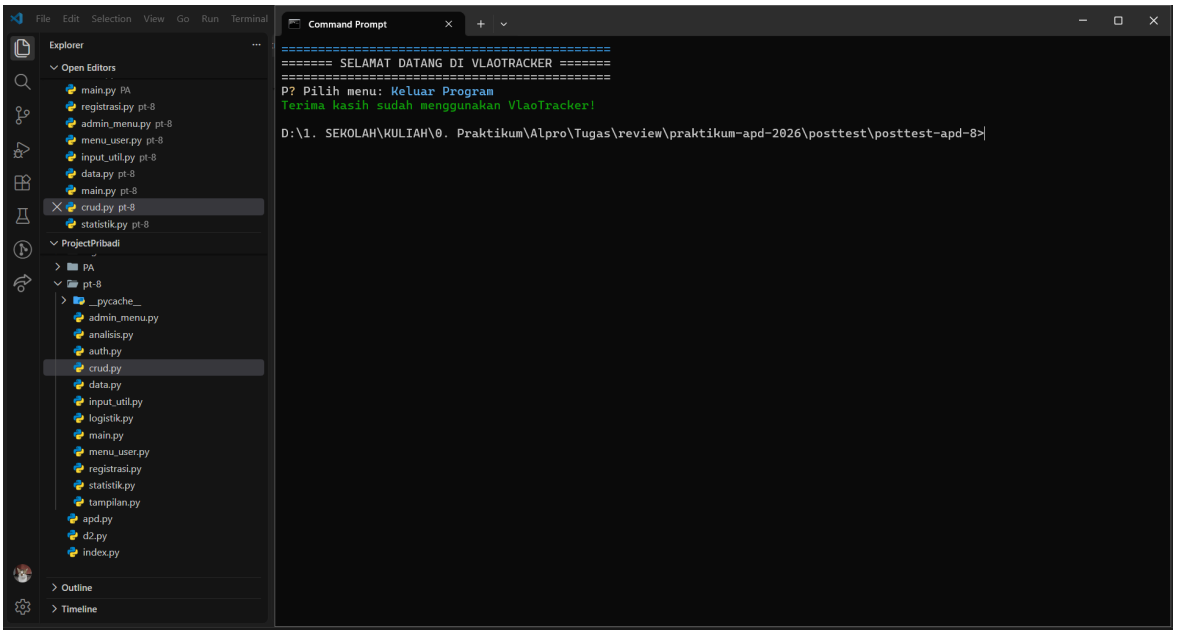
Gambar 4.14 Screenshot Output Terminal Delete



Gambar 4.15 Screenshot Output Terminal Rekapulasi Stat

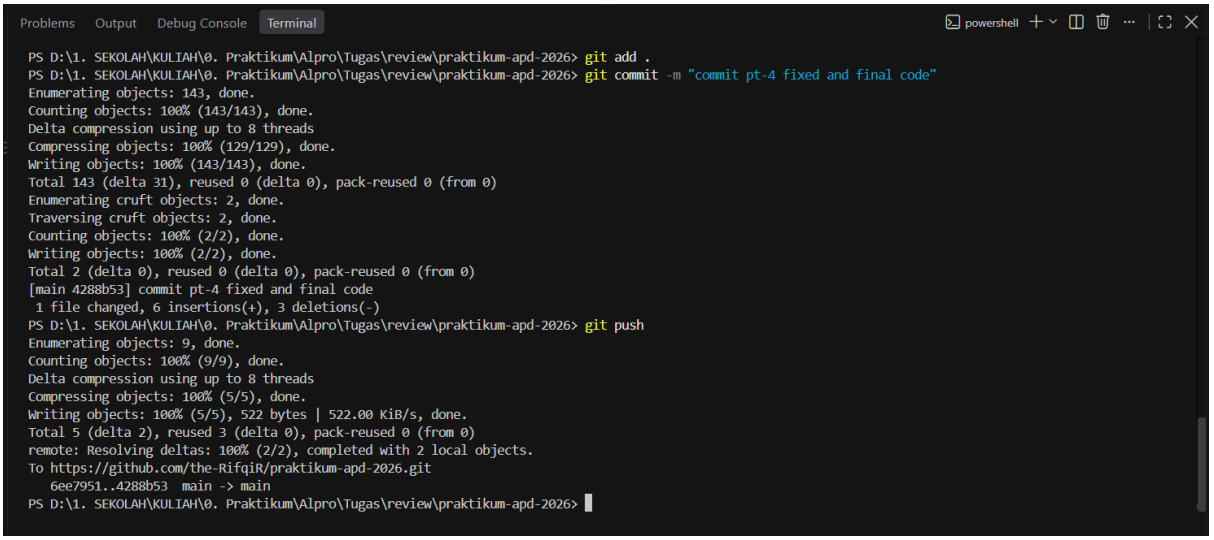


Gambar 4.16 Screenshot Output Terminal Rekap Admin Setelah Ada Data



Gambar 4.17 Screenshot Output Terminal Out Program

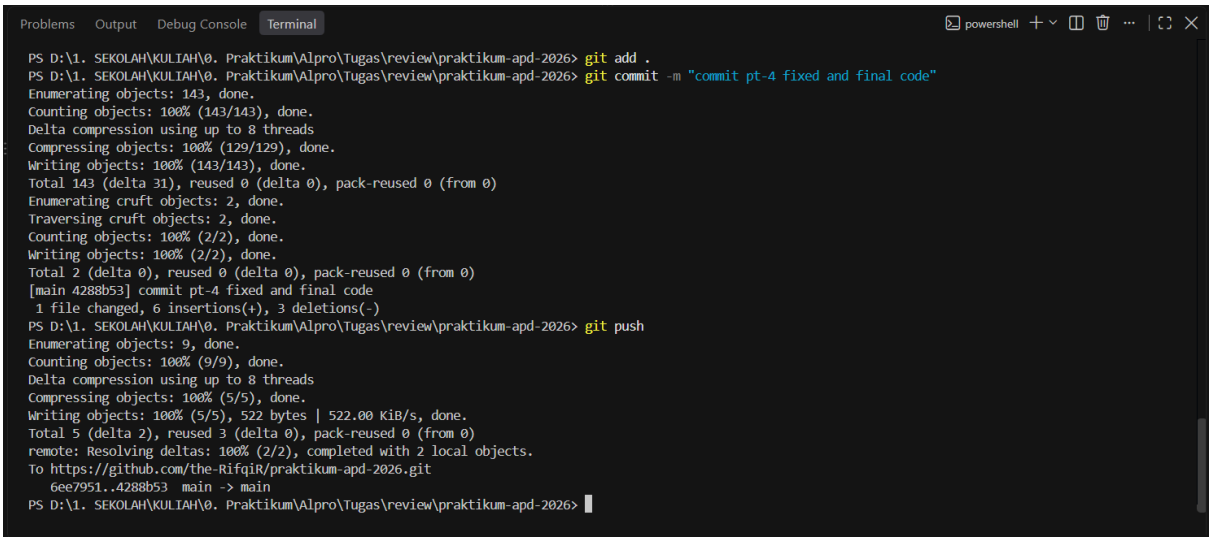
5. Git Add



Gambar 5.1 Screenshot Output Terminal Perintah git add

Penjelasannya sesuai nama “add” artinya menambahkan kalau git add . menambahkan semua perubahan ke dalam antrian git.

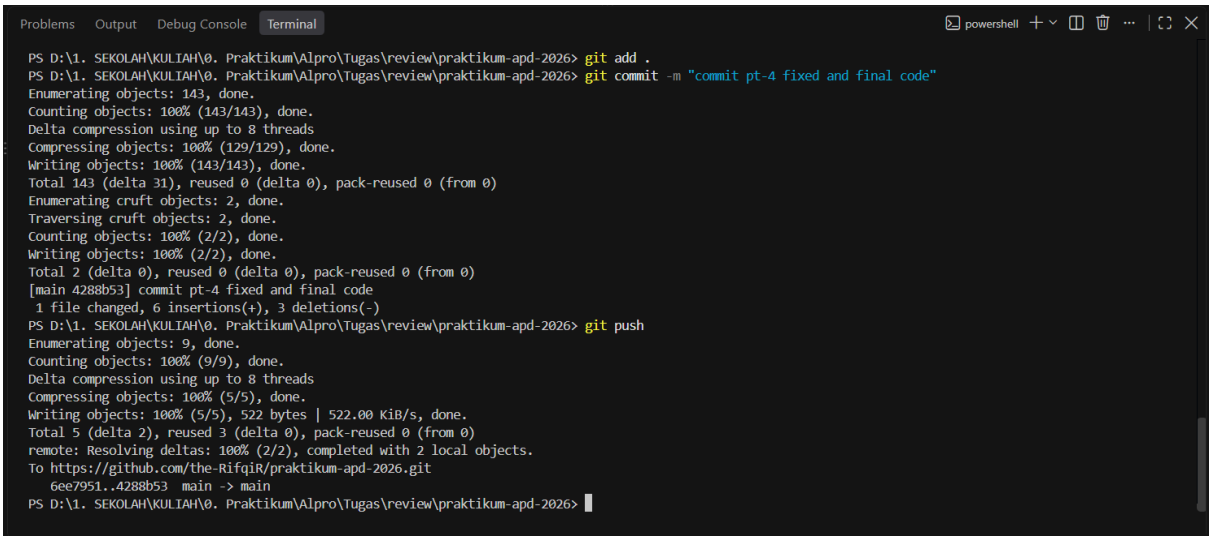
6. Git Commit



Gambar 5.2 Screenshot Output Terminal Perintah git commit

Commit di git buat nandain perubahan yang akan dikirim kegithub dengan memberi inisial commitnya.

7. Git Push



Gambar 5.3 Screenshot Output Terminal Perintah git push

Git push, mengirimkan data yang sudah ditambahkan dan diinisialisasikan ke github.